CPU ve GPU - Tanımlar, Çalışma Mantığı, Farklar ve Örnekler

---

1. Önkoşullar (Prerequisites)

Bu ders içeriğini ve teknik dokümanı eksiksiz bir şekilde takip edebilmek ve donanım-yazılım etkileşimini kavramak için bilgisayar mühendisliği öğrencilerinin aşağıdaki temel ön bilgilere sahip olması beklenmektedir:

- **Temel Bilgisayar Organizasyonu:** İşlemci çekirdeği yapısı, bellek hiyerarşisi (yazmaçlar, L1/L2/L3 önbellek seviyeleri, ana bellek), komut işleme çevrimi (*fetch-decode-execute*) ve sistem veri yolları (*bus*) hakkında kavramsal yetkinlik.
- **Temel Kodlama ve Algoritma Mantığı:** C veya C++ dillerinde dizi (*array*), çok boyutlu matris işlemleri, döngü yapıları (**for**, **while**) ve koşullu dallanma ifadeleri (**if-else**) içeren kod bloklarını okuma, analiz etme ve karmaşıklık boyutlarını değerlendirebilme becerisi.
- **Heterojen Hesaplama (Heterogeneous Computing) Modeli:** Sistem mimarisinin iki temel mantıksal ve fiziksel bileşenden oluştuğunun kavranması:
  - **Host (Sunucu / Ev Sahibi):** İşletim sistemini çalıştıran, genel amaçlı seri kodları yürüten, karmaşık kontrol mantığını yöneten ve ana sistem belleğinden sorumlu bir veya birkaç **CPU (Central Processing Unit / Merkezi İşlem Birimi: Bilgisayar sisteminin genel amaçlı komutlarını yürüten ve mantıksal denetimi sağlayan ana işlemci)** (Intel CPU vs. GPU). CPU, bu süreçte **RAM (Random Access Memory / Rastgele Erişimli Bellek: CPU'nun işlediği veri ve komutları geçici olarak tutan yüksek hızlı ana sistem belleği)** bileşenini doğrudan yönetir.
  - **Device (Aygıt):** Yoğun paralel hesaplama yüklerini üstlenmek üzere yapılandırılmış, kendi yüksek bant genişlikli fiziksel belleğine sahip bir veya birden fazla **GPU (Graphics Processing Unit / Grafik İşlem Birimi: Yüksek derecede paralelleştirilebilir veri hesaplama iş yüklerini yürüten özel işlemci)**. GPU, hesaplamalar esnasında **VRAM (Video Random Access Memory / Video Rastgele Erişimli Bellek: GPU'nun yüksek bant genişliğiyle eriştiği aygıt belleği)** birimini kullanır.

---

2. Tanımlar (Definitions: CPU ve GPU)

2.1 CPU (Central Processing Unit / Merkezi İşlem Birimi)

CPU, genel amaçlı bir bilgisayar sisteminin komut işleme merkezi olup geleneksel olarak "bilgisayarın beyni" olarak adlandırılır (Intel CPU vs. GPU). Milyarlarca transistörden meydana gelen CPU mimarisi, sınırlı sayıda (örneğin 4 ila 64 arasında) fakat her biri son derece yüksek saat frekanslarında çalışan, gelişmiş komut seviyesinde paralellik (*instruction-level parallelism*) sunan karmaşık çekirdeklerden (*cores*) oluşur.

Bir CPU çekirdeğinin temel donanım mimarisi şu ana bileşenleri barındırır:

- **ALU (Arithmetic Logic Unit / Aritmetik Mantık Birimi: Matematiksel ve mantıksal işlemleri yürüten donanım bloğu):** Toplama, çıkarma, kaydırma ve mantıksal Karşılaştırma operasyonlarını yüksek hassasiyetle gerçekleştiren temel hesaplama birimi.

- **Kontrol Birimi (Control Unit):** Komutları bellekten getiren (*fetch*), kodunu çözen (*decode*), sıralı veya sırasız yürütmeyi (*out-of-order execution*) organize eden ve dallanma tahmincilerini (*branch predictors*) yöneten denetim mekanizması.
- **Yazmaçlar (Registers: İşlemci çekirdeği içinde yer alan ve komutların en düşük gecikmeyle erişebildiği en hızlı bellek hücreleri):** İşlemci çevrim süresiyle (0-1 saat döngüsü) eşzamanlı çalışabilen en üst seviye bellek katmanı.
- **Önbellek (Cache):** Ana bellek erişim gecikmesini maskelemek amacıyla kullanılan, donanımsal olarak yönetilen L1, L2 ve L3 seviyelerindeki yüksek hızlı yerel bellekler.

CPU'nun temel mimari amacı; işletim sistemi süreçlerini kesintisiz yürütmek, tekil çekirdek performansını (*per-core performance*) zirveye çıkarmak ve seri iş yüklerinde gecikmeyi (**Latency: Tek bir görevin veya komutun tamamlanması için geçen süre**) en aza indirmektir.

2.2 GPU (Graphics Processing Unit / Grafik İşlem Birimi)

GPU, tarihsel gelişim sürecinde 3D grafik oluşturma ve piksel işleme süreçlerini hızlandırmak amacıyla geliştirilmiş sabit işlevli bir **ASIC (Application-Specific Integrated Circuit / Uygulamaya Özel Entegre Devre: Belirli bir algoritmayı çalıştırmak üzere özel olarak tasarlanmış entegre devre)** yapısından evrilmiştir (Intel CPU vs. GPU). Zamanla bu sabit yapılar programlanabilir gölgelendiricilere (*programmable shaders*) ve en nihayetinde genel amaçlı paralel hesaplama işlemcilerine (GPGPU) dönüşmüştür.

GPU mimarisi, CPU'ya kıyasla daha basit denetim birimlerine ve küçük önbelleklere sahip, ancak sayıları binleri (örneğin 2048 ile 16384 arasında) bulan özelleşmiş küçük çekirdek bloklarından oluşur. GPU'nun temel donanım amacı, veri kümelerini çok sayıda alt parçaya bölerek aynı anda işlemek ve birim zamanda aktarılan/işlenen toplam veri miktarını, yani işleme hacmini (**Throughput: Birim zamanda tamamlanan toplam iş veya aktarılan veri miktarı**) en üst seviyeye çıkarmaktır.

2.3 Özel Donanım Bileşenleri

- **NPU (Neural Processing Unit / Nöral İşlem Birimi: Yapay zeka ve derin öğrenme çıkarım süreçlerini donanımsal olarak hızlandıran özel işlem birimi):** Derin yapay zeka ağlarının matris çarpımı ve aktivasyon fonksiyonu gibi spesifik matris matris operasyonlarını düşük güç tüketimiyle yürütmek için tasarlanmış bileşendir (Intel CPU vs. GPU). NPU, yapay zeka çıkarım (*inferencing*) aşamalarında CPU ve GPU üzerindeki işlem yükünü hafifleterek güç verimliliği sağlar.

---

3. Çalışma Mantığı Adımları (Operating Logic Steps)

Bir yazılım sisteminde heterojen veri işleme süreci CPU ve GPU arasında belirli adımlarla yürütülür (NVIDIA CUDA Best Practices Guide):

```
[Adım 1: CPU Seri Yürütme ve Karar Mantığı]
   └── Karmaşık Kontrol, OS Komutları, Dallanma Tahmini
         │
[Adım 2: Host-Device Veri Aktarımı (PCIe Transferi)]
   └── RAM'den VRAM'e Veri Taşıma (APOD Döngüsü)
         │
[Adım 3: GPU Paralelleştirme (SIMT ve Warp Organizasyonu)]
   └── 32'lik Warp Gruplarında Hafif İş Parçacıkları (Threads)
```

```
        |
[Adım 4: Hizalanmış Bellek Erişimi ve Yürütme]
   └── 32-Baytlık Segmentler Halinde Bitişik (Coalesced) Erişim
```

Adım 1: Komut Getirme ve Seri Yürütme (CPU)

CPU, yazılımın ana akışını kontrol eder. Komut seviyesindeki karmaşık mantık denetimi, işletim sistemi çağrıları ve yüksek oranda dallanma içeren doğrusal kodlar CPU çekirdeklerinde yürütülür. Çekirdek içerisindeki gelişmiş dallanma tahmincileri ve derin önbellek hiyerarşisi, veri erişim gecikmelerini gizlemek üzere sürekli çalışır.

Adım 2: İş Yükünü Ayrıştırma (Host-Device Paylaşımı)

Uygulamanın profil analizi yapıldığında, yoğun hesaplama gerektiren ve paralelleştirilmeye uygun olan kod blokları (*hotspots*) tespit edilir. **APOD (Assess, Parallelize, Optimize, Deploy / Değerlendir, Paralelleştir, Eniyile, Yayına Al: Yazılımları GPU üzerinde aşamalı olarak hızlandırma döngüsü)** tasarım yöntemi uyarınca, veri dizilimleri ana bellekten (RAM) aygıt belleğine (VRAM) **PCIe (Peripheral Component Interconnect Express / Çevresel Bileşen Bağlantı Ekspresi: Ana sistem ile aygıtlar arasında veri aktarımı sağlayan yüksek hızlı veri yolu)** arabirimi üzerinden aktarılır (NVIDIA CUDA Best Practices Guide).

Adım 3: İş Parçacığı (Thread) Organizasyonu ve Paralelleştirme

- **CPU İş Parçacıkları:** Ağır sıklettir (*heavyweight*). Bir CPU iş parçacığından diğerine geçiş yapıldığında işletim sistemi seviyesinde bağlam değişimi (*context switch*) gerçekleşir; yazmaçlar ve durum bilgileri ana belleğe saklandığı için bu işlem yüksek saat döngüsü maliyetine yol açar. Bu nedenle CPU'da az sayıda iş parçacığı yönetilir.
- **GPU İş Parçacıkları:** Hafif sıklettir (*lightweight*). GPU mimarisinde donanımsal yazmaç dosyaları (*register file*) aktif olan tüm iş parçacıklarına fiziksel olarak dağıtılır. Bu sayede bir iş parçacığı grubundan diğerine geçiş sıfır gecikme maliyetiyle gerçekleşir. GPU üzerinde iş parçacıkları **Warp** adı verilen 32'şerli gruplar halinde organize edilerek donanımsal yürütme birimlerine sunulur (NVIDIA CUDA Best Practices Guide).

Adım 4: Bellek Erişimi ve Bitişik Erişim (Coalescing)

GPU çekirdekleri veriyi VRAM (Global Memory), L2 ve L1 önbellekler üzerinden okur. Bir Warp içindeki 32 iş parçacığı ardışık bellek adreslerine eriştiğinde donanım bu erişimleri tek bir bellek işleminde birleştirir. Buna **Bitişik Bellek Erişimi (Coalesced Memory Access)** denir. Modern GPU mimarilerinde bellek erişim birimi 32-baytlık hizalanmış segmentlerden oluşur; Warp içindeki her bir iş parçacığının bitişik 4-baytlık tek duyarlıklı veriye erişmesi durumunda toplam 128 baytlık veri talebi, 4 adet 32-baytlık bellek işlemine dönüştürülerek maksimum bant genişliği elde edilir (NVIDIA CUDA Best Practices Guide).

Mimari Kavram Tanımlamaları

- **SIMD (Single Instruction, Multiple Data / Tek Komut, Çoklu Veri: Tek bir vektör komutunun birden fazla veri elemanı üzerinde aynı anda çalıştırılmasını sağlayan mimari yapı):** CPU'lardaki AVX veya NEON gibi vektör birimlerinde tek bir komut paletinin geniş vektör yazmaçlarındaki çoklu veriye uygulanmasıdır.
- **SIMT (Single Instruction, Multiple Threads / Tek Komut, Çoklu İş Parçacığı: Eşzamanlı çalışan birden fazla iş parçacığının aynı komutu farklı veriler üzerinde**

yürütmesini sağlayan GPU yürütme modeli): GPU'larda kullanılan temel yürütme mimarisidir (NVIDIA CUDA Best Practices Guide). SIMD'den farklı olarak, SIMT modelinde her iş parçacığı kendi yazmaç adreslemesine, kendi komut işaretçisine ve bağımsız dallanma durumuna sahiptir; ancak donanım seviyesinde 32 iş parçacığı tek bir Warp oluşturarak aynı komutu eşzamanlı yürütür.

- **Gecikme (Latency) vs. İşleme Hacmi (Throughput):**
  - *Gecikme (Latency):* Tek bir işin veya veri paketinin kaynaktan hedefe ulaşması için geçen mutlak süredir. CPU mimarileri gecikmeyi minimize etmek için devasa önbellekler ve gelişmiş mantık denetim sistemleri kullanır.
  - *İşleme Hacmi (Throughput):* Belirli bir zaman aralığında tamamlanan toplam iş miktarıdır. GPU mimarileri, yüksek gecikmeyi binlerce iş parçacığı arasında donanımsal geçiş yaparak maskeler ve işleme hacmini maksimuma ulaştırır.
  - *Mühendislik Analojisi:* Latency odaklı CPU, bir yolcuyu A noktasından B noktasına 300 km/s hızla taşıyan tek kişilik bir spor otomobildir. Throughput odaklı GPU ise 100 km/s hızla giden ancak aynı anda 500 yolcuyu taşıyan bir yolcu trenidir. Tek bir yolcu için otomobil hızlı görünse de, 10.000 yolcunun taşınması gerektiğinde tren toplam sürede otomobili katlarca geride bırakır.

---

4. Somut Örnekler (Concrete Examples)

Örnek 1: Matris Çarpımı (C = A x B) ve GPU Bellek Kullanımı

Varsayımlar: N x N boyutunda, tek duyarlıklı (32-bit float / 4 bayt) elemanlardan oluşan iki boyutlu matrisler. Hesaplama boyutu **N = 4096** olarak alınmıştır. Matematiksel işlem yükü **2 x (N üzeri 3)** yani yaklaşık 137.4 milyar kayar nokta işlemidir (FLOP). Veri boyutu ise 3 matris için toplam **3 x (N üzeri 2) x 4 bayt = 201.3 Megabayttır (MB)**.

İşlem / Transfer Oranı (Compute-to-Transfer Ratio) Analizi: Matris toplama işleminde (**C = A + B**), transfer edilen her eleman için yalnızca 1 toplama yapıldığı için İşlem/Transfer oranı **O(1)** seviyesindedir. Bu tür düşük oranlı iş yüklerinde PCIe transfer maliyeti GPU hesaplama kazancını tamamen yok edebilir. Oysa matris çarpımında (**C = A x B**) transfer edilen **O(N üzeri 2)** elemana karşılık **O(N üzeri 3)** matematiksel işlem gerçekleştirilir. İşlem/Transfer oranı **O(N)** olduğu için, veri kopyalama maliyeti yüksek hesaplama hızı altında kolayca tolere edilir (NVIDIA CUDA Best Practices Guide).

Adım Adım Akış ve Bant Genişliği Evrimi:

1. **Bellek Tahsisi ve Transfer:** Host CPU, RAM üzerinde A ve B matrislerini oluşturur. **cudaMalloc** ile VRAM üzerinde yer açılır. Veri, PCIe Gen3 x16 arabirimi üzerinden (teorik bant genişliği ~16 GB/s, pratik pinned memory ile ~12 GB/s) GPU küresel belleğine (Global Memory) kopyalanır.
2. **Izgara (Grid) ve Blok Yapılandırması:** Hesaplama **TILE_DIM x TILE_DIM** boyutunda (örneğin **32 x 32 = 1024** iş parçacığı içeren) bloklara bölünür. Izgara boyutu **(N/32) x (N/32)** bloktan oluşur. Her iş parçacığı C matrisinin bir elemanını hesaplamakla görevlendirilir.
3. **Optimize Edilmemiş Doğrudan Erişim (Yalnızca Global Memory):** İş parçacıkları A matrisinin satırlarını ve B matrisinin sütunlarını doğrudan küresel bellekten okursa, A

matrisinin aynı elemanları tekrar tekrar okunur. NVIDIA Tesla V100 GPU üzerinde yapılan ölçümlerde bu ham erişim yöntemi yalnızca **119.9 GB/s** efektif bant genişliği sunmaktadır (NVIDIA CUDA Best Practices Guide).

4. **Paylaşımlı Bellek (Shared Memory) Kullanımı ve Karolama (Tiling):** A matrisinin karoları çip üzerindeki düşük gecikmeli Paylaşımlı Belleğe (**__shared__ float aTile[32][32]**) bitişik olarak kopyalanır. Sadece A matrisinin paylaşımlı belleğe alınmasıyla efektif bant genişliği **144.4 GB/s** seviyesine yükselir. B matrisinin de karolar halinde paylaşımlı belleğe yüklenmesi ve **__syncthreads()** ile blok içi senkronizasyon sağlanması sonucunda, gereksiz küresel bellek okumaları engellenir ve efektif bant genişliği **195.5 GB/s** seviyesine ulaşır (NVIDIA CUDA Best Practices Guide).
5. **Sonucun Geri Aktarılması:** Hesaplanan C matrisi VRAM'den PCIe otobüsü aracılığıyla Host RAM'e kopyalanır ve bellek serbest bırakılır.

Matris Çarpımı Zaman ve Bellek Katmanı Şeması:

Aşama	İşlem / Veri Akışı	Kullanılan Bellek Türü	Efektif Bant Genişliği / Maliyet
Aşama 1	Verinin Host RAM -> Device VRAM Aktarımı	PCIe Bus -> Global Memory	~12 - 16 GB/s (Darboğaz Aşaması)
Aşama 2 (Ham)	Doğrudan Küresel Bellek Okumasıyla Çarpım	Global Memory -> Register	119.9 GB/s (Yüksek Önbellek Tahsisi)
Aşama 3 (Karo A)	A Matris Karosunun Paylaşımlı Belleğe Alınması	Global Memory -> Shared Memory	144.4 GB/s (Tekrarlı Okuma Azaltıldı)
Aşama 4 (Karo A+B)	A ve B Karolarının Paylaşımlı Belleğe Alınması	Shared Memory -> Register	195.5 GB/s (Maksimum Başarım)
Aşama 5	Sonucun VRAM -> Host RAM Aktarımı	Global Memory -> PCIe Bus -> RAM	~12 - 16 GB/s

Örnek 2: CPU-GPU İş Paylaşımı (Render, Tarayıcı ve Kontrol Mantığı)

Varsayımlar: Modern bir web tarayıcı motorunun veya kullanıcı arayüzü yazılımının iş parçacığı dağılımı.

Adım Adım Akış:

1. **CPU (Kontrol ve Karar Mantığı):** CPU; kullanıcı etkileşimlerini (fare hareketleri, klavye girdileri) olay döngüsü (*event loop*) içinde dinler. DOM ağacını oluşturur, CSS kurallarını çözümler, ağ isteklerini (HTTP/HTTPS) ve dosya I/O işlemlerini yönetir. Bu süreçler doğrusal denetim ve yüksek oranda **if-else** dallanması içerdiği için CPU üzerinde yürütülür.
2. **Veri Hazırlığı ve Komut Listesi Oluşturma:** CPU, ekrana çizilecek elemanların koordinatlarını, kaplama (*texture*) verilerini ve vektörel dönüşüm matrislerini hazırlar. Bu verileri aygıt belleğine aktarır ve GPU'nun anlayacağı çizim komut dizisini (*draw calls*) hazırlar.

3. **GPU (Piksel İşleme ve Render):** GPU, kendisine iletilen koordinatları ve kaplamaları alarak piksel renklendirme (*rasterization*), kompozitleme, gölgeleme ve geometrik dönüşüm işlemlerini binlerce çekirdeğinde eşzamanlı olarak işler ve kare arabelleğine (*framebuffer*) yazarak ekrana basar.

---

5. Karşılaştırma (Comparison)

CPU ve GPU Temel Mimarisi Karşılaştırma Tablosu

Parametre	CPU (Central Processing Unit)	GPU (Graphics Processing Unit)
Donanım Amacı	Genel amaçlı seri komut işleme ve sistem yönetimi	Karmaşık ve yoğun paralel veri işleme
Çekirdek Sayısı ve Tipi	Az sayıda (örn. 4 - 64), karmaşık ve yüksek frekanslı	Binlerce (örn. 2048 - 16384), basit ve özel
İş Parçacığı Yapısı	Ağır sıklet (*Heavyweight*), yüksek değişim maliyeti	Hafif sıklet (*Lightweight*), sıfır değişim maliyeti
Birlikte Çalıştığı Bellek	Sistem Belleği (**RAM**)	Grafik / Aygıt Belleği (**VRAM**)
Temel Odak Noktası	Düşük Gecikme (**Latency**)	Yüksek İşleme Hacmi (**Throughput**)
Paralellik Tipi	Görev Paralelliği (*Task Parallelism*)	Veri Paralelliği (*Data Parallelism*)
Bellek Bant Genişliği	Tipik olarak 50 - 200 GB/s (DDR4/DDR5)	Tipik olarak 500 - 2000 GB/s (GDDR6/HBM2)

Çekirdek Karşılaştırması Uyarısı

Bir CPU çekirdeği ile bir GPU çekirdeği bire bir niceliksel olarak kıyaslanamaz (Intel CPU vs. GPU). Bir CPU çekirdeği; karmaşık denetim birimlerine, agresif dallanma tahmincilerine, komut seviyesinde paralellik (ILP) sağlayan mekanizmalara ve GHz seviyesinde yüksek saat frekanslarına sahiptir. Bir GPU çekirdeği ise temel olarak matematiksel komutları yürüten basit bir ALU ve yazmaç grubundan ibarettir. Modern CPU'lar çoklu çekirdek yapıları ve AVX-512 gibi SIMD vektör komut setleriyle kendi içlerinde paralel çalışabilse de, toplam veri paralelliği kapasiteleri GPU mimarilerinin gerisindedir.

Entegre (iGPU) ve Harici (dGPU) Grafik Ayrımı

- **Entegre Grafik İşlem Birimi (iGPU - Integrated GPU: CPU ile aynı çip üzerinde yer alan ve sistem RAM'ini paylaşımlı kullanan grafik birimi):** CPU ile aynı silikon paket üzerinde bulunur (Intel CPU vs. GPU). Fiziksel olarak ayrı bir VRAM bellek hattı bulunmaz; sistem belleğini (RAM) CPU ile ortaklaşa kullanır.
  - *Zero-Copy Avantajı:* iGPU sistemlerinde CPU ve GPU belleği fiziksel olarak aynı DRAM'i paylaştığı için **Zero-Copy (Sıfır Kopyalama / Mapped Pinned Memory)** tekniği kullanılabilir (NVIDIA CUDA Best Practices Guide). Bu sayede PCIe kopyalama maliyeti tamamen ortadan kalkar ve GPU doğrudan sistem belleğindeki veriye erişir.

- **Harici Grafik İşlem Birimi (dGPU - Discrete GPU: Bağımsız VRAM'e sahip, PCIe üzerinden bağlanan grafik kartı):** Anakarta PCIe veri yolu üzerinden bağlanan bağımsız bir kart üzerindedir (Intel CPU vs. GPU). Yüksek watt değerlerinde çalışır ve kendi üzerinde ultra yüksek bant genişlikli VRAM (GDDR6, HBM2) barındırır. Maksimum paralel hesaplama performansı sunar.
  - **UVA (Unified Virtual Addressing / Birleşik Sanal Adresleme: CPU ana belleği ile GPU aygıt belleklerini tek bir sanal adres alanında birleştiren bellek mimarisi):** Modern 64-bit sistemlerde UVA sayesinde Host belleği ve tüm dGPU bellekleri tek bir sanal adres uzayında birleştirilir; böylece işaretçilerin fiziksel konumu sürücü katmanında şeffaf olarak yönetilir (NVIDIA CUDA Best Practices Guide).

Yazılım Ortamı ve Platform Sınırlamaları

- **CUDA:** NVIDIA tarafından geliştirilen, geliştiricilerin C/C++ dilleriyle GPU üzerinde doğrudan hesaplama yapmasını sağlayan, yalnızca NVIDIA donanım mimarilerine özel bir paralel programlama mimarisi ve uygulama ortamıdır (NVIDIA CUDA Best Practices Guide).
- **API (Application Programming Interface / Uygulama Programlama Arayüzü: Yazılımların donanım sürücüleriyle haberleşmesini sağlayan standart arayüz):** Yazılımların donanım sürücüleriyle konuşmasını sağlayan soyutlama katmanıdır. CUDA platforma ve üreticiye bağımlı bir API/mimari iken, Vulkan, OpenCL veya DirectCompute gibi açık standart API'lar farklı donanım üreticilerinin (Intel, AMD, NVIDIA) işlemcileri üzerinde çalışabilirlik sunar.

---

6. Hata ve Sınırlamalar (Errors & Limitations)

GPU mimarisi paralel hesaplamalarda muazzam bir güç sunsa da belirli mühendislik sınırlamalarına ve performans darboğazlarına tabidir:

1. Veri Aktarım Maliyeti (PCIe Darboğazı)

Sistem belleğinden (RAM) grafik belleğine (VRAM) yapılan kopyalamalar PCIe arabirimi üzerinden gerçekleşir. PCIe Gen3 x16 veri yolunun pratik bant genişliği **~12-16 GB/s** civarındayken, aygıt içi VRAM bant genişliği (örneğin NVIDIA Tesla V100 üzerindeki HBM2 bellek ile) **898 GB/s** seviyesindedir (NVIDIA CUDA Best Practices Guide).

Eğer aktarılan veri boyutu yüksek fakat veriler üzerinde yapılacak matematiksel işlem sayısı az ise (düşük İşlem/Transfer oranı), veri taşıma süresi GPU'da kazanılan hesaplama süresini aşar ve sistem toplamda CPU yürütmesinden daha yavaş çalışır.

2. Dallanma Sapması (Branch Divergence)

Bir Warp içindeki 32 iş parçacığı tek bir komut birimi tarafından yönlendirilir (SIMT). Kod içerisinde bir **if-else** koşulu sebebiyle Warp içindeki iş parçacıklarının bir kısmı **if** bloğuna, diğer kısmı **else** bloğuna yönelirse **Dallanma Sapması** meydana gelir (NVIDIA CUDA Best Practices Guide).

Bu durumda donanım iki yürütme kolunu aynı anda çalıştıramaz. Önce **if** bloğundaki iş parçacıkları çalıştırılırken **else** kolundaki iş parçacıkları pasif kılınarak bekletilir (*branch predication*); ardından **else** bloğu yürütülür. İşlemler serileştiği için yürütme maliyeti iki katına çıkar ve GPU performansı ciddi oranda düşer.

```
Warp (32 İş Parçacığı)
```

```
    |
    ├──> [Koşul Değerlendirmesi: if (threadIdx.x % 2 == 0)]
    |       ├── 16 İş Parçacığı -> IF Bloğu   (Seri Yürütme 1 - ELSE kolu pasif bekler)
    |       └── 16 İş Parçacığı -> ELSE Bloğu (Seri Yürütme 2 - IF kolu pasif bekler)
    |
    └──> [Kesişim Noktası ve Tekrar Birleşme (Re-convergence)]
```

3. Hizalanmamış ve Düzensiz Bellek Erişimi (Uncoalesced Access & Bank Conflicts)

- **Uncoalesced Access:** Warp içindeki iş parçacıkları bellekteki ardışık olmayan adreslere eriştiğinde (örneğin adımlı/strided erişim), donanım tek bir 32-baytlık segment yerine çok sayıda ayrı bellek işlemi gerçekleştirmek zorunda kalır ve bant genişliği heder olur (NVIDIA CUDA Best Practices Guide).
- **Bank Conflicts (Banka Çatışmaları):** Çip içi Paylaşımlı Bellek (Shared Memory), eşzamanlı erişim sağlamak için 32 bağımsız bellek bankasına (*memory banks*) bölünmüştür (NVIDIA CUDA Best Practices Guide). Bir Warp içindeki birden fazla iş parçacığı aynı bellek bankasındaki farklı adreslere erişmeye çalışırsa banka çatışması oluşur ve erişimler serileşir.
  - *Çözüm Teknikleri:* Transpoze matris çarpımlarında (**C = A x A^T**), matris sütun bazlı yazıldığında 32-way banka çatışması yaşanır ve performans 140.2 GB/s seviyesinde kalır. Paylaşımlı bellek tanımına 1 elemanlık dolgu eklenerek (**__shared__ float transposedTile[32][33];**), modüler aritmetikle banka indeksleri birer birer kaydırılır. Çatışmalar tamamen sıfırlandığında efektif bant genişliği **199.4 GB/s** seviyesine ulaşır (NVIDIA CUDA Best Practices Guide).

4. Amdahl Kanunu ve Güçlü Ölçekleme Sınırlaması

Amdahl Kanunu, bir yazılımın paralelleştirilemeyen seri kısmının (**1 - P**), sisteme ne kadar paralel çekirdek eklenirse eklensin elde edilebilecek maksimum teorik hızlanmayı (**S**) sınırlandırdığını ifade eder (NVIDIA CUDA Best Practices Guide).

Düz metin formülü: **S = 1 / ((1 - P) + (P / N))**

Burada:

- **S**: Sistem genelinde elde edilen hızlanma oranı (*Speedup*).
- **P**: Kodun paralelleştirilebilir kısmının oranı (**0 <= P <= 1**).
- **1 - P**: Kodun paralelleştirilemeyen seri kısmının oranı.
- **N**: Paralel yürütmeyi sağlayan işlemci/çekirdek sayısı.

İşlemci sayısı **N** sonsuza doğru yaklaştığında (**P / N** terimi sıfıra yaklaşır), formül şu teorik üste dönüşür: **S_max = 1 / (1 - P)**

Mühendislik Hesaplama Örneği: Bir algoritmanın kod diziliminin %80'i paralelleştirilebiliyorsa (**P = 0.80**), seri kalan kısım %20'dir (**1 - P = 0.20**). Bu sistemde 10.000 çekirdeğe sahip devasa bir GPU kullanılsa dahi ulaşılabilecek maksimum teorik hızlanma sınırı **1 / 0.20 = 5** kat ile sınırlıdır. Seri kod bölümü optimize edilmeden donanım eklemek performansı artırmaz.

---

7. Sözlük (Glossary)

Terim / Kısaltma	İngilizce Açılımı	Türkçe Anlamı ve Kısa İşlev Açıklaması
ALU	Arithmetic Logic Unit	**Aritmetik Mantık Birimi:** Matematiksel ve mantıksal işlemleri yürüten donanım bloğu.
API	Application Programming Interface	**Uygulama Programlama Arayüzü:** Yazılımların sürücü ve donanım katmanıyla haberleşmesini sağlayan standart arayüz.
APOD	Assess, Parallelize, Optimize, Deploy	**Değerlendir, Paralelleştir, Eniyile, Yayına Al:** GPU yazılım geliştirme ve iyileştirme döngüsü.
ASIC	Application-Specific Integrated Circuit	**Uygulamaya Özel Entegre Devre:** Belirli bir görevi veya algoritmayı çalıştırmak üzere özel tasarlanmış çip.
CPU	Central Processing Unit	**Merkezi İşlem Birimi:** Sistem komutlarını ve işletim sistemini yürüten genel amaçlı ana işlemci.
CUDA	Compute Unified Device Architecture	NVIDIA donanımlarına özgü paralel hesaplama mimarisi ve yazılım geliştirme ortamı.
dGPU	Discrete Graphics Processing Unit	**Harici Grafik İşlem Birimi:** PCIe arabirimiyle bağlanan, bağımsız VRAM'e sahip yüksek performanslı grafik kartı.
Host / Device	Host (CPU+RAM) / Device (GPU+VRAM)	Heterojen sistemlerde ana bilgisayar (Host) ile hesaplama aygıtı (Device) mimari ayrımı.
iGPU	Integrated Graphics Processing Unit	**Entegre Grafik İşlem Birimi:** CPU ile aynı silikonda yer alan ve sistem RAM'ini paylaşımlı kullanan grafik birimi.
Latency	Latency	**Gecikme:** Tek bir işlemin veya komutun tamamlanması için geçen mutlak süre.
NPU	Neural Processing Unit	**Nöral İşlem Birimi:** Yapay zeka ve derin öğrenme çıkarım süreçlerini hızlandıran özel donanım bloğu.
NUMA	Non-Uniform Memory Access	**Düzensiz Bellek Erişimi:** Bellek erişim süresinin işlemcinin bellek modülüne olan fiziksel uzaklığına göre değiştiği mimari.
RAM	Random Access Memory	**Rastgele Erişimli Bellek:** CPU'nun işlediği veri ve komutları geçici tutan ana sistem belleği.

Terim / Kısaltma	İngilizce Açılımı	Türkçe Anlamı ve Kısa İşlev Açıklaması
Register	Register	**Yazmaç:** İşlemci çekirdeği içinde yer alan, sıfır gecikmeye yakın en hızlı bellek hücresi.
SIMD	Single Instruction, Multiple Data	**Tek Komut, Çoklu Veri:** Vektörel veriler üzerinde tek komutla paralel işlem yapma mimarisi.
SIMT	Single Instruction, Multiple Threads	**Tek Komut, Çoklu İş Parçacığı:** GPU'larda birden fazla iş parçacığının aynı komutu farklı verilerle yürütme modeli.
Throughput	Throughput	**İşleme Hacmi:** Birim zamanda tamamlanan toplam iş veya aktarılan veri miktarı.
UVA	Unified Virtual Addressing	**Birleşik Sanal Adresleme:** Host ve Device belleklerini tek bir sanal adres alanında birleştiren bellek mimarisi.
VRAM	Video Random Access Memory	**Video Rastgele Erişimli Bellek:** GPU'ya doğrudan bağlı, ultra yüksek bant genişlikli aygıt belleği.
Warp	Warp	GPU üzerinde aynı anda eşzamanlı donanımsal olarak yürütülen 32'lik iş parçacığı grubu.

---

8. Açıklamalı Kontrol Sorusu (Self-Assessment Questions)

Soru 1: CPU ve GPU çekirdekleri arasındaki mimari ve kullanım amacı farkı nedir?

Yanıt: CPU çekirdekleri, karmaşık kontrol birimlerine, gelişmiş önbelleklere, dallanma tahmincilerine ve yüksek saat frekanslarına sahiptir; temel amaçları seri kod yürütmede tekil çekirdek performansını maksimuma çıkarmak ve gecikmeyi (Latency) düşürmektir (Intel CPU vs. GPU). GPU çekirdekleri ise basit yapıda ALU bloklarından oluşur ve binlerce sayıda bir arada bulunur; temel amaçları büyük veri kümelerini paralelleştirerek birim zamandaki toplam işleme hacmini (Throughput) zirveye çıkarmaktır.

Soru 2: Latency (Gecikme) ve Throughput (İşleme Hacmi) kavramlarını bellek erişim ve hesaplama performansları açısından karşılaştırınız.

Yanıt: Latency, tek bir bellek okuma veya komut yürütme işleminin başlatılması ile tamamlanması arasında geçen mutlak süredir (örneğin 100 nanosaniye). Throughput ise birim zamanda bellekten çekilen veya işlenen toplam veri miktarıdır (örneğin saniyede 800 Gigabayt). CPU mimarileri Latency odaklı olup önbelleklerle bu süreyi düşürmeyi hedefler; GPU mimarileri ise Throughput odaklı olup yüksek gecikmeyi binlerce iş parçacığı arasında donanımsal geçiş yaparak maskeler.

Soru 3: PCIe üzerinden yapılan veri transferlerinin GPU hesaplama performansına etkisi nedir ve İşlem/Transfer oranı bunu nasıl şekillendirir?

Yanıt: PCIe otobüsünün bant genişliği (~12-16 GB/s), VRAM'in kendi iç bant genişliğine (~898 GB/s) kıyasla çok düşüktür (NVIDIA CUDA Best Practices Guide). Aktarılan veri üzerinde yapılan matematiksel işlem sayısı az ise (düşük İşlem/Transfer oranı, örn. matris toplama **O(1)**), veri taşıma süresi GPU hesaplama kazancını yok eder. Yapılan işlem sayısı yüksekse (örn. matris çarpımı **O(N)**), kopyalama maliyeti tolere edilir ve GPU yüksek başarım sağlar.

Soru 4: Amdahl Kanunu'na göre, bir kodun %15'lik kısmı paralelleştirilemiyorsa (seri kalıyorsa), 5000 çekirdekli bir GPU sisteminde elde edilebilecek maksimum teorik hızlanma nedir?

Yanıt: Amdahl Kanunu formülünde **1 - P = 0.15**, **P = 0.85** ve **N = 5000** yazılırsa: **S = 1 / (0.15 + (0.85 / 5000)) = 1 / (0.15 + 0.00017) = 1 / 0.15017 = 6.65** Sisteme 5000 çekirdek eklense dahi maksimum hızlanma teorik olarak 6.66 katı geçemez. Kodun %15'lik seri kısmı temel performans darboğazını belirler (NVIDIA CUDA Best Practices Guide).

Soru 5: CUDA platformunda geliştirilmiş bir yazılım Intel veya AMD GPU donanımları üzerinde doğrudan çalıştırılabilir mi?

Yanıt: Çalıştırılamaz. CUDA, NVIDIA'nın kendi donanım ve sürücü mimarisine özel geliştirdiği kapalı bir platformdur (NVIDIA CUDA Best Practices Guide). Farklı üreticilerin GPU'ları üzerinde ortak kod çalıştırabilmek için platform bağımsız standart API'ların (Vulkan, OpenCL vb.) tercih edilmesi gerekir.

Soru 6: Entegre grafik birimi (iGPU) ile harici grafik kartı (dGPU) bellek kullanımı ve veri kopyalama maliyetleri açısından nasıl ayrışır?

Yanıt: iGPU, fiziksel olarak ayrı bir VRAM'e sahip değildir; sistem ana belleğini (RAM) CPU ile ortaklaşa kullanır. Bu sayede Zero-Copy (Sıfır Kopyalama) tekniğiyle PCIe kopyalama maliyeti olmadan veri paylaşımı yapar. dGPU ise kendi kartı üzerinde ultra yüksek hızlı bağımsız VRAM'e sahiptir; veriler PCIe üzerinden kopyalanmak zorundadır ancak VRAM içi erişim hızları iGPU'ya kıyasla katlarca yüksektir (NVIDIA CUDA Best Practices Guide).

Soru 7: Bir Warp içinde yer alan 32 iş parçacığının 16'sı if bloğuna, diğer 16'sı else bloğuna girdiğinde GPU yürütme biriminde ne gerçekleşir?

Yanıt: Dallanma Sapması (*Branch Divergence*) oluşur. SIMT mimarisi gereği Warp içindeki iş parçacıkları aynı komutu yürütmek zorundadır. Donanım önce **if** bloğundaki 16 iş parçacığını çalıştırırken **else** kolundakileri pasif tutar; ardından **else** bloğundaki 16 iş parçacığını çalıştırıp **if** kolundakileri pasif tutar. İşlemler serileştiği için yürütme süresi iki katına çıkar (NVIDIA CUDA Best Practices Guide).

Soru 8: Paylaşımlı Bellek (Shared Memory) kullanımı ve banka padding tekniği, küresel bellek darboğazını ve banka çatışmalarını nasıl çözer?

Yanıt: Küresel bellek çip dışı (*off-chip*) DRAM üzerindedir ve erişim gecikmesi yüksektir. Paylaşımlı bellek ise çip içi (*on-chip*) L1 seviyesinde ultra hızlı bir bellektir. Veriler bir kez küresel bellekten okunup paylaşımlı belleğe yazıldığında sonraki okumalar çip içinden yapılır ve küresel bellek darboğazı çözülür (NVIDIA CUDA Best Practices Guide). Paylaşımlı bellekteki 32 banka arasındaki çakışmaları önlemek için dizi boyutuna 1 elemanlık dolgu (**padding**) eklenerek modüler aritmetikle banka indeksleri kaydırılır ve çakışmalar engellenir.

---

9. Kaynaklar (References)

1. **Intel Corporation**, *CPU vs. GPU: What's the Difference?*

URL: **https://www.intel.com/content/www/us/en/products/docs/processors/cpu-vs-gpu.html**

2. **NVIDIA Corporation**, *CUDA C++ Best Practices Guide*

URL: **https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html**